

ASSIGNMENT 6:

NAME : DIYA RAVINDRA BHANSALI
PRN : 125B1F271
CLASS : SY BTECH IT

INTERFACE

/* 6) Create an application for a transportation system where different modes of transport (e.g., car, bus, train) demonstrate polymorphism in their pricing strategies. Implement a common "Transport" interface with a polymorphic method for calculating fares, and each transport type overrides this method to apply its specific fare calculation logic.

*/

```
import java.util.*;
```

```
// Interface is a completely abstract blueprint - it contains declarations without any implementation
```

```
// Interfaces have attributes as type public, static, final
```

```
interface Transport {  
    double calculateFare(double distance);  
}
```

```
// Car class
```

```
class Car implements Transport {  
    public double calculateFare(double distance) {  
        double baseFare = 50;  
        double farePerKm = 15;  
        return baseFare + (distance * farePerKm);  
    }  
}
```

```
// Bus class
```

```
class Bus implements Transport {  
    String type;  
    Bus(String type) {  
        this.type = type;  
    }  
    public double calculateFare(double distance) {  
        double baseFare;  
        double farePerKm;  
        if (type.equals("AC")) {  
            baseFare = 30;  
            farePerKm = 10;  
        }  
    }  
}
```

```

        else {
            baseFare = 20;
            farePerKm = 5;
        }
        return baseFare + (distance * farePerKm);
    }
}

```

// Train class

```

class Train implements Transport {
    String type;
    Train(String type) {
        this.type = type;
    }
    public double calculateFare(double distance) {
        double baseFare;
        double farePerKm;
        if (type.equals("First Class")) {
            baseFare = 100;
            farePerKm = 10;
        }
        else {
            baseFare = 30;
            farePerKm = 5;
        }
        return baseFare + (distance * farePerKm);
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Input
        System.out.print("Enter distance in km: ");
        double distance = sc.nextDouble();
        System.out.print("Enter number of persons travelling: ");
        int numberOfPersons = sc.nextInt();

        // Create objects
        Transport car = new Car();
        Transport acBus = new Bus("AC");
        Transport nonAcBus = new Bus("Non-AC");
        Transport firstClassTrain = new Train("First Class");
        Transport generalTrain = new Train("General");

        // Calculate fare per person
    }
}

```

```

double carFare = car.calculateFare(distance);
double acBusFare = acBus.calculateFare(distance);
double nonAcBusFare = nonAcBus.calculateFare(distance);
double firstClassTrainFare = firstClassTrain.calculateFare(distance);
double generalTrainFare = generalTrain.calculateFare(distance);
// Calculate total fare
double carTotal = carFare * numberOfPersons;
double acBusTotal = acBusFare * numberOfPersons;
double nonAcBusTotal = nonAcBusFare * numberOfPersons;
double firstClassTrainTotal = firstClassTrainFare * numberOfPersons;
double generalTrainTotal = generalTrainFare * numberOfPersons;

// Display
System.out.println("\n===== TRAVEL FARE COMPARISON =====");
System.out.println("\nCar");
System.out.println("Fare Per Person: " + carFare);
System.out.println("Total Fare: " + carTotal);
System.out.println("\nBus - AC");
System.out.println("Fare Per Person: " + acBusFare);
System.out.println("Total Fare: " + acBusTotal);
System.out.println("\nBus - Non-AC");
System.out.println("Fare Per Person: " + nonAcBusFare);
System.out.println("Total Fare: " + nonAcBusTotal);
System.out.println("\nTrain - First Class");
System.out.println("Fare Per Person: " + firstClassTrainFare);
System.out.println("Total Fare: " + firstClassTrainTotal);
System.out.println("\nTrain - General");
System.out.println("Fare Per Person: " + generalTrainFare);
System.out.println("Total Fare: " + generalTrainTotal);

double minimumFare = carTotal;
String bestOption = "Car";
if (acBusTotal < minimumFare) {
    minimumFare = acBusTotal;
    bestOption = "AC Bus";
}
if (nonAcBusTotal < minimumFare) {
    minimumFare = nonAcBusTotal;
    bestOption = "Non-AC Bus";
}
if (firstClassTrainTotal < minimumFare) {
    minimumFare = firstClassTrainTotal;
    bestOption = "First Class Train";
}
if (generalTrainTotal < minimumFare) {
    minimumFare = generalTrainTotal;

```

```

        bestOption = "General Train";
    }
    // Display
    System.out.println("\n===== BEST OPTION =====");
    System.out.println("Best Transport: " + bestOption);
    System.out.println("Minimum Total Fare: " + minimumFare);
    System.out.println("Number of Persons: " + numberOfPersons);
    System.out.println("Distance: " + distance + " km");
}
}

```

OUTPUT:

```

C:\Users\DIYA\Desktop>java Main
Enter distance in km: 23
Enter number of persons travelling: 3

===== TRAVEL FARE COMPARISON =====

Car
Fare Per Person: 395.0
Total Fare: 1185.0

Bus - AC
Fare Per Person: 260.0
Total Fare: 780.0

Bus - Non-AC
Fare Per Person: 135.0
Total Fare: 405.0

Train - First Class
Fare Per Person: 330.0
Total Fare: 990.0

Train - General
Fare Per Person: 145.0
Total Fare: 435.0

===== BEST OPTION =====
Best Transport: Non-AC Bus
Minimum Total Fare: 405.0
Number of Persons: 3
Distance: 23.0 km

C:\Users\DIYA\Desktop>

```

Assignment 6: AI-Generated Code & Comparison

Name: Diya Ravindra Bhansali **PRN:** 125B1F271 **Class:** SY BTech IT

Problem Statement

Create an application for a transportation system where different modes of transport (car, bus, train) demonstrate polymorphism in their pricing strategies. Implement a common "Transport" interface with a polymorphic method for calculating fares, and each transport type overrides this method to apply its specific fare calculation logic.

AI-Generated Code

The following implementation solves the same problem using an abstract base class, boolean flags instead of string types, and a collection-driven loop instead of repeated variables.

```
import java.util.*;

// Interface defining the fare calculation contract
interface Transport {
    double calculateFare(double distance);
    String getName();
}

// Abstract base class to reduce duplication across transport types
abstract class AbstractTransport implements Transport {
    protected double baseFare;
    protected double farePerKm;
    protected String name;

    public double calculateFare(double distance) {
        return baseFare + (distance * farePerKm);
    }

    public String getName() {
        return name;
    }
}

class Car extends AbstractTransport {
    Car() {
        this.name = "Car";
        this.baseFare = 50;
        this.farePerKm = 15;
    }
}

class Bus extends AbstractTransport {
    Bus(boolean isAC) {
        this.name = isAC ? "Bus - AC" : "Bus - Non-AC";
        this.baseFare = isAC ? 30 : 20;
        this.farePerKm = isAC ? 10 : 5;
    }
}

class Train extends AbstractTransport {
    Train(boolean isFirstClass) {
        this.name = isFirstClass ? "Train - First Class" : "Train - General";
        this.baseFare = isFirstClass ? 100 : 30;
        this.farePerKm = isFirstClass ? 10 : 5;
    }
}
```

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter distance in km: ");
        double distance = sc.nextDouble();
        System.out.print("Enter number of persons travelling: ");
        int persons = sc.nextInt();

        List<Transport> options = new ArrayList<>();
        options.add(new Car());
        options.add(new Bus(true));
        options.add(new Bus(false));
        options.add(new Train(true));
        options.add(new Train(false));

        System.out.println("\n===== TRAVEL FARE COMPARISON =====");

        Transport best = null;
        double minTotal = Double.MAX_VALUE;

        for (Transport t : options) {
            double perPerson = t.calculateFare(distance);
            double total = perPerson * persons;

            System.out.println("\n" + t.getName());
            System.out.println("Fare Per Person: " + perPerson);
            System.out.println("Total Fare: " + total);

            if (total < minTotal) {
                minTotal = total;
                best = t;
            }
        }

        System.out.println("\n===== BEST OPTION =====");
        System.out.println("Best Transport: " + best.getName());
        System.out.println("Minimum Total Fare: " + minTotal);
        System.out.println("Number of Persons: " + persons);
        System.out.println("Distance: " + distance + " km");
    }
}

```

Comparison: Original Code vs AI-Generated Code

Aspect	Your Code	AI-Generated Code
Structure	Transport interface implemented directly by Car, Bus, Train	Adds an intermediate AbstractTransport class implementing Transport; Car, Bus, Train extend it instead

Code duplication	Each class repeats its own calculateFare logic and field declarations	calculateFare() is written once in the abstract class; subclasses only set field values in their constructors
Sub-type distinction	Uses a String type field (e.g., "AC", "First Class") checked with .equals()	Uses a boolean flag (e.g., isAC, isFirstClass) — avoids string comparison and typos
Managing multiple objects	Each transport object declared and processed with separate variables (carFare, acBusFare, etc.)	All transport objects stored in a List<Transport> and processed in a loop
Finding best option	A chain of if statements comparing each total individually	A single loop that tracks the minimum while iterating
Extensibility	Adding a new transport type or fare tier means writing a new class from scratch and adding new if blocks	Adding a new type just means creating a small subclass of AbstractTransport; comparison logic needs no changes
Polymorphism demonstrated	Yes — interface reference calling overridden method	Yes, and additionally demonstrated through the loop, where the same calculateFare()/getName() calls work for any Transport without knowing its concrete type at compile time
Readability for small programs	Straightforward, very explicit, easy to trace line by line	Slightly more abstract; requires understanding the abstract class before understanding subclasses

Key Takeaway

Both implementations correctly demonstrate polymorphism via the Transport interface — that requirement is met either way. The AI-generated version trades a bit of directness for reduced duplication and easier scalability (using an abstract class, boolean flags, and a collection-driven loop), which matters more as the number of transport types grows. For a small, fixed set of five options like this assignment, the original approach is perfectly reasonable and arguably easier to read for a grader seeing it for the first time.